

OpenVela 工程实践

# 多模态感知智能打地鼠竞技系统

## 开题报告

SmartMole Pro • OpenVela / NuttX

开发平台 DshanPI openvela Devkit • T113S3

指导教师 修佳鹏

提交日期 2026 年 06 月 14 日

### 项目成员

张恒基

2024211301

2024210926

郭志罡

2024211302

2024210937

曹佳轩

2024211312

2024210994

张耀辉

2024211302

2024211045

缪钰

2024211303

2024210996

朱辰骏

2024211301

2024211039

OpenVela 嵌入式工程实践

## 摘要

SmartMole Pro 是以基础实验 WhackMole 为起点，在 OpenVela (NuttX RTOS) 平台上开发的嵌入式娱乐系统。本项目在保留多模态输入、声光反馈与数据持久化扩展能力的同时，聚焦两大核心玩法：五级闯关与 Wi-Fi 双板联机——每台开发板独立全屏运行，不做同屏分屏。

闯关模式提供 LEVEL 1 - 5 共五关，每关通过独立参数表配置地鼠刷新闻隔、停留时长、同屏数量与移动速度，形成由易到难的梯度挑战。联机模式下两台 DshanPI 开发板经 Wi-Fi 点对点连接，同步 START / SCORE / FINISH 等状态，在相同关卡配置下实时比拼得分。系统支持触摸屏、物理按键与 HC-SR04 挥击等多模态输入，经统一打击事件队列驱动 LVGL 游戏逻辑；WS2812B 灯环与音效实现关卡切换、连击与结算等声光联动；littlefs 本地存储支撑排行榜与关卡进度。

本开题报告阐述 SmartMole Pro 项目的背景、系统目标、技术架构、模块设计、采购方案、人员分工、开发计划及预期难点，为后续工程实施提供总体方案依据。具体实施进度见《任务进度报告》，阶段分工见《分工报告》。

关键词：SmartMole Pro；OpenVela；NuttX；闯关模式；关卡难度；Wi-Fi 联机

# 目录

|                                          |    |
|------------------------------------------|----|
| 一、引言                                     | 1  |
| 1.1 项目背景                                 | 1  |
| 1.2 openvela 平台概述                        | 1  |
| 1.3 报告结构                                 | 1  |
| 二、系统目标                                   | 1  |
| 2.1 项目定位                                 | 1  |
| 2.2 分层目标体系                               | 2  |
| 2.3 功能优先级                                | 2  |
| 三、系统架构                                   | 2  |
| 3.1 总体架构                                 | 2  |
| 3.2 软件分层说明                               | 3  |
| 3.3 多线程设计                                | 3  |
| 3.4 统一打击事件队列                             | 4  |
| 四、开发环境搭建                                 | 4  |
| 4.1 硬件与软件环境                              | 4  |
| 4.2 编译与烧录                                | 5  |
| 4.3 基础外设验证                               | 5  |
| 五、功能模块设计                                 | 6  |
| 5.1 多模态输入系统                              | 6  |
| 5.2 五级闯关与关卡难度                            | 6  |
| 5.3 AI 动态难度（可选扩展）                        | 7  |
| 5.4 Wi-Fi 双板联机                           | 7  |
| 5.4.1 联机通信协议                             | 7  |
| 5.5 声光反馈系统                               | 8  |
| 5.6 数据存储与成就系统                            | 9  |
| 5.7 特殊目标逻辑                               | 9  |
| 六、采购方案                                   | 9  |
| 七、人员分工                                   | 10 |
| 7.1 团队成员                                 | 10 |
| 7.2 张恒基 — 系统总架构 + 全局集成 + 联机联调 + 项目统筹     | 11 |
| 7.3 曹佳轩 — 多模态输入系统 + 事件队列 + 硬件驱动          | 11 |
| 7.4 缪钰 — 声光反馈 + GUI 全面重构 + 全模块调试         | 12 |
| 7.5 郭志罡 — 关卡难度体系 + 特殊地鼠 + 优化触屏逻辑         | 12 |
| 7.6 张耀辉 — Wi-Fi 联机 + versus 模块           | 12 |
| 7.7 朱辰骏 — 数据存储 + WiFi 连接可视化 + 排行榜 + 关卡数据 | 13 |
| 八、项目计划                                   | 13 |
| 九、预期难点与应对                                | 14 |
| 9.1 多线程资源竞争                              | 14 |
| 9.2 模块并行集成冲突                             | 14 |
| 9.3 WS2812B 时序精度                         | 14 |
| 十、反转模式（加分项，可选）                           | 15 |
| 10.1 玩法设计                                | 15 |
| 10.2 传感器逻辑                               | 15 |
| 10.3 得分机制                                | 15 |
| 10.4 联机扩展（可选）                            | 15 |
| 十一、开题答辩 Q&A 与技术决策                        | 16 |
| 11.1 关于系统目标                              | 16 |
| 11.2 关于系统架构                              | 16 |
| 11.3 关于采购方案                              | 17 |
| 11.4 关于人员分工                              | 18 |
| 11.5 关于预期难点                              | 18 |
| 11.6 关于反转模式                              | 18 |
| 11.7 硬件降级策略汇总                            | 19 |
| 十二、总结                                    | 19 |
| 十三、参考文献                                  | 20 |
| 十四、音效反馈系统实现                              | 20 |
| 14.1 音频资源清单                              | 20 |

|                             |    |
|-----------------------------|----|
| 14.2 技术实现                   | 20 |
| 十五、LED 反馈实现与 WS2812B 降级决策   | 21 |
| 15.1 实际实现                   | 21 |
| 15.2 降级原因                   | 21 |
| 十六、Wi-Fi 双板联机实现细节           | 22 |
| 16.1 协议栈与模块文件               | 22 |
| 16.2 线程与 UI 同步              | 22 |
| 16.3 Wi-Fi 图形连接 (wifi_ui.c) | 22 |
| 16.4 IP 锁定问题分析              | 22 |
| 十七、未实现功能分析                  | 23 |
| 17.1 HC-SR04 超声波挥击          | 23 |
| 17.2 WS2812B 全彩灯环           | 23 |
| 17.3 littlefs 排行榜与成就        | 23 |
| 17.4 AI 动态难度                | 23 |
| 17.5 音效全场景挂钩                | 24 |
| 17.6 反转模式                   | 24 |

# 一、引言

## 1.1 项目背景

嵌入式系统课程的基础实验 WhackMole 实现了触摸屏点击打地鼠的单机小游戏，验证了 LCD 显示、触摸输入与简单游戏逻辑在 NuttX RTOS 上的可行性。然而，基础实验在输入方式、反馈体验、难度策略、社交竞技与数据留存等方面均存在明显局限，难以体现现代嵌入式产品的工程完整度。

SmartMole Pro 项目旨在以 WhackMole 为起点，在 OpenVela 平台上完成一次系统性的功能扩展与工程升级：从“能玩”到“好玩”，从“单机演示”到“可答辩的完整产品”。OpenVela 基于 Apache NuttX 内核，具备 88% POSIX 兼容性与轻量化 AIoT 能力，是本项目的操作系统基座。开发硬件选用全志 T113S3 平台的 DshanPI openvela Devkit，屏幕分辨率 480×320，配套 LED、按键、触摸屏、音频与 Wi-Fi 等外设。

## 1.2 openvela 平台概述

根据《openvela 快速入门与工程实践》手册，NuttX、Xiaomi Vela 与 openvela 三者关系为：NuttX 提供实时操作系统内核；Xiaomi Vela 是小米基于 NuttX 定制的商业物联网平台；openvela 则是 Vela 核心代码的 Apache 2.0 开源社区版本。平台具备系统内核、驱动框架、AI 框架、LWAC 框架与维测套件五大关键能力，高度可扩展，适用于从 32KB RAM 模组到 512MB RAM 智能音箱的多种硬件规格。

## 1.3 报告结构

第二章至第十一章为开题方案正文；任务进度与阶段分工另见独立文档 `progress.typ`、`division.typ`。

# 二、系统目标

## 2.1 项目定位

本项目以基础实验 WhackMole 为起点，在 OpenVela (NuttX RTOS) 平台上，将单机触控小游戏扩展为以五级闯关与 Wi-Fi 双板联机为核心的嵌入式娱乐系统：单机逐关挑战，或两台开发板各自全屏、远程同步对战，每关难度参数独立可配。

## 2.2 分层目标体系

| 目标层级    | 具体描述                          |
|---------|-------------------------------|
| 基 础 目 标 | 完整复现 WhackMole 基础游戏，驱动稳定运行    |
| 扩 展 目 标 | 实现物理按键 + 悬空感应（超声波）的多模态输入融合    |
| 进 阶 目 标 | Wi-Fi 双板联机对战 + 五级闯关（每关独立难度参数） |
| 亮 点 目 标 | WS2812B 声光联动 + 本地排行榜与关卡进度持久化  |
| 工 程 目 标 | 完成外壳集成，以完整产品形态参加答辩            |

## 2.3 功能优先级

项目按“必做”与“加分”两档规划，确保四周周期内答辩可控：

必做项（第二周末完成）： 五级闯关可玩 + 物理按键 + 音效 + 本地排行榜  
加分项（第三周冲刺）： 超声波挥击 + WS2812B 灯效 + Wi-Fi 双板联机稳定演示  
联机采用双板各一块屏、各自全屏方案，不做 LVGL 同屏分屏。即使加分项未能全部完成，必做项已超过基础实验要求。

# 三、系统架构

## 3.1 总体架构

系统采用四层架构，自顶向下分别为用户交互层、NuttX RTOS 应用层、驱动与外设层，各层通过统一打击事件队列解耦。



## 3.2 软件分层说明

| 层次    | 职责                                             |
|-------|------------------------------------------------|
| 操作系统层 | NuttX RTOS，负责多任务调度、设备节点管理与 POSIX IPC           |
| 图形层   | LVGL，横屏 480×320，负责所有 UI 渲染与动画                  |
| 游戏逻辑层 | 状态机驱动（待机→游戏中→结算），管理所有游戏对象                      |
| 多线程模型 | GUI 主线程 / 音频线程 / 传感器轮询线程 / 数据存储线程，通过消息队列和互斥锁协同 |

## 3.3 多线程设计

| 线程       | 优先级 | 职责                            |
|----------|-----|-------------------------------|
| LVGL 主线程 | 最高  | UI 渲染与游戏主逻辑，mq_receive 处理打击事件 |
| 音频线程     | 次高  | 背景音乐、打击音效、连击特效音叠加混音           |
| 传感器轮询    | 最低  | 50ms 轮询 HC-SR04，检测挥击动作        |

| 线程     | 优先级 | 职责                       |
|--------|-----|--------------------------|
| 数据存储线程 | 低   | 游戏结束时写入 littlefs，不占用实时资源 |

Kconfig 中栈空间配置为 327680 字节。共享游戏状态变量使用 `pthread_mutex` 保护，打击事件通过 POSIX 消息队列传递，避免竞态条件。

### 3.4 统一打击事件队列

三路输入汇聚为同一数据结构，上层游戏逻辑无需区分来源：

```
typedef enum {
    HIT_SRC_TOUCH,
    HIT_SRC_BUTTON,
    HIT_SRC_ULTRASONIC
} hit_source_t;

typedef struct {
    hit_source_t source;
    int x, y;
} hit_event_t;
```

- 触摸屏：直接读取触点坐标；
- 物理按键：K1 映射左半区中心，K2 映射右半区中心；
- 超声波：检测到距离突变（100ms 内从大于 15cm 降至小于 10cm）后，填入当前地鼠位置坐标。

各输入处理函数构造 `hit_event_t` 后调用 `mq_send` 写入同一消息队列，游戏主逻辑通过 `mq_receive` 取事件处理。

## 四、开发环境搭建

本章内容参照《openvela 快速入门与工程实践（基于 T113S3）》手册。

### 4.1 硬件与软件环境

| 类别  | 配置                                     |
|-----|----------------------------------------|
| 开发板 | DshanPI openvela Devkit (T113S3)，实验室提供 |



| 类别   | 配置                                                 |
|------|----------------------------------------------------|
| 操作系统 | Ubuntu 虚拟机，用户名/密码：ubuntu                           |
| IDE  | VS Code + C/C++ 插件，源码路径 /home/book/vela-opensource |
| 串口调试 | MobaXterm，波特率 1500000，Flow Control: none           |
| 烧录工具 | PhoenixSuit 全志线刷工具                                 |

## 4.2 编译与烧录

```
cd ~/vela-opensource/vendor/allwinnertech/lichee/
source vela_env.sh && source envsetup.sh
lunch_nuttx
m && pack
```

烧录流程：安装 UsbDriver → PhoenixSuit 加载镜像并勾选“全盘擦除升级” → 按住 FEL 短按 RST 进入烧写模式。

## 4.3 基础外设验证

在集成 SmartMole Pro 新外设之前，需确认开发板基础功能正常：

| 模块     | 命令                          | 验证结果            |
|--------|-----------------------------|-----------------|
| LED/按键 | nsh> led                    | LED1 闪烁，K1 触发点亮 |
| 触摸屏    | nsh> ft5x06 /<br>dev/input0 | 打印触点坐标          |
| LCD    | nsh> fb                     | 320×480 矩形色块显示  |
| 音频     | aplay /data/<br>moon.wav    | 正常播放测试音频        |
| Wi-Fi  | ifup wlan0 +<br>wapi        | 连接 AP 并 ping 外网 |

### 注意

串口流控必须设为 none；音频播放中避免 Ctrl+C 中断，否则需重启才能再次播放。

## 五、功能模块设计

### 5.1 多模态输入系统

玩家可通过三种方式打地鼠，体验各不相同：

1. 触摸屏点击 — 与基础实验一致，手指直接点击地鼠；
2. 物理大按键 — 左键打屏幕左半区，右键打右半区，适合快速连击；
3. 悬空挥击 — HC-SR04 检测手部距离，挥击时距离从约 25cm 突变至 8cm 以内触发打击。

#### 说明

HC-SR04 精度  $\pm 3\text{mm}$ ，量程 2 - 400cm。软件滤波要求 100ms 内距离从大于 15cm 降至小于 10cm 才算有效打击，区分自然挥击与缓慢靠近，有效抑制误触发。

按键采用硬件 + 软件双重防抖，支持短按、长按、连击多种事件类型。联机模式下各板独立采集输入，经 versus 模块同步分数与对局状态，无需同屏分屏。

### 5.2 五级闯关与关卡难度

与基础实验“全局统一加速”不同，本系统将难度按关卡切分：LEVEL 1 - 5 各维护一张参数表，字段包括地鼠刷新间隔、停留时长、同屏最大数量、基础移动速度及炸弹/黄金地鼠概率等。玩家逐关挑战，通关后解锁下一关；联机时双方加载同一 LEVEL 配置，在各自全屏界面上比拼得分。

| 关卡 | 难度倾向 | 典型参数变化            |
|----|------|-------------------|
| L1 | 入门   | 刷新慢、停留长、单鼠为主      |
| L2 | 轻松   | 间隔缩短，开始出现双鼠       |
| L3 | 标准   | 速度与数量均衡提升         |
| L4 | 紧张   | 停留缩短，炸弹概率上升       |
| L5 | 极限   | 高频刷新 + 多鼠同屏 + 短停留 |

参数表由 `MyWhackMole.c` 中 LEVEL 宏或外置配置加载，后续可扩展 AI 在关卡表基础上微调，但答辩保底以固定五关差异化难度为准。

5.3 AI 动态难度（可选扩展）

在固定关卡表之上，可选启用 AI 引擎：每 5 秒采样命中率、反应时间与连击数，在当前关卡参数附近小幅调节间隔与停留时长。该模块为加分项，不影响“每关基础难度不同”的核心设计。

5.4 Wi-Fi 双板联机

不做同屏分屏。联机采用两台开发板各自独立全屏运行，经 Wi-Fi 点对点同步对局。MyWhackMole.c 中 versus 模块已验证以下能力：

|       |                                        |
|-------|----------------------------------------|
| 功能    | 实现方案                                   |
| 双板全屏  | 每块板各自 LVGL 全屏界面，无左右分屏                  |
| 状态同步  | START / SCORE / FINISH 广播，MODE 切换单人/联机 |
| 关卡一致  | 联机开局指定同一 LEVEL，共享难度参数表                 |
| 特殊地鼠  | 黄金 +5、炸弹 -2、COMBO 与联机计分兼容              |
| IP 配置 | 联机 IP 宏外置，换板仅改配置不改 versus 核心           |

说明

联机模块由张耀辉与朱辰骏分工完成：张耀辉负责 versus 核心实现（R528 UDP 框架、协议与 IP/端口配置）；朱辰骏参与双板联调与实机测试，提供一块开发板，排查 IP 配置、UDP 通信、状态不同步等问题。

5.4.1 联机通信协议

联机不使用 HTTP、MQTT、WebSocket 等重量级协议，而是在 OpenVela/NuttX 原生 TCP/IP 协议栈之上，由 MyWhackMole.c 的 versus 模块实现轻量应用层报文。板内多线程仍用 POSIX 消息队列传递 hit\_event\_t，与板间 Wi-Fi 协议相互独立。

|       |                   |                                                 |
|-------|-------------------|-------------------------------------------------|
| 层次    | 技术                | 说明                                              |
| 物理/链路 | IEEE 802.11 Wi-Fi | 开发板 STA 模式接入同一无线 AP，局域网互通                       |
| 网络    | IPv4              | 双板各配置固定 IP，当前宏写死，后续外置到配置文件                      |
| 传输    | UDP 数据报           | POSIX socket + sendto/recvfrom，低延迟、无连接，适合分数实时同步 |

| 层次 | 技术           | 说明                                              |
|----|--------------|-------------------------------------------------|
| 应用 | Versus 自定义报文 | START / SCORE / FINISH / MODE 四类消息，versus 状态机解析 |

应用层消息语义：

1. START — 一方发起对局，携带 LEVEL 等参数，双方同步开局倒计时；
2. SCORE — 周期性或事件触发上报本机得分、连击等，驱动对手界面刷新；
3. FINISH — 对局结束，携带最终比分，触发结算页与排行榜写入；
4. MODE — 切换单机闯关 / Wi-Fi 联机，与主菜单 MODE 框架一致。

```
typedef enum {
    VS_MSG_START,
    VS_MSG_SCORE,
    VS_MSG_FINISH,
    VS_MSG_MODE,
} versus_msg_type_t;

// 报文经 UDP 发送至对端固定 IP:PORT
// 具体端口与 payload 布局以 MyWhackMole.c 宏定义为准
```

说明

选型理由：联机仅需少量状态同步，UDP 延迟低于 TCP，丢包可由 SCORE 周期性重发或 FINISH 最终态兜底；versus 核心已验收，后续仅允许改 IP/端口宏，不改报文语义与状态机。

5.5 声光反馈系统

WS2812B 8 灯环通过 SPI 模拟时序驱动（2.4MHz 时钟，1-bit 映射为 3-bit SPI 波形），灯控在独立线程运行，与 LVGL 解耦。

| 游戏状态  | 灯效       | 音效      |
|-------|----------|---------|
| 待机    | 蓝色呼吸     | 背景音乐    |
| 普通击中  | 单灯白色闪烁   | hit.wav |
| 连击 ≥3 | 橙色流光     | 连击专属音效  |
| 关卡切换  | 短促蓝白闪烁   | 升级提示音   |
| 游戏结束  | 彩虹爆闪 3 秒 | 结算播报    |

| 游戏状态       | 灯效 | 音效   |
|------------|----|------|
| 倒计时<br>≤5s | —  | 语音提醒 |

若 WS2812B 驱动不稳定，降级为板载 LED1/LED2/LED3 简化灯效，切换仅需改一个宏定义，游戏逻辑不受影响。

## 5.6 数据存储与成就系统

基于 littlefs 文件系统实现多类型数据持久化，含掉电保护、数据校验与并发读写优化：

1. 关卡通关记录、每关最高分、最高连击、联机对战胜负、游戏时长；
2. 连击、通关、联机胜率、特殊目标触发等多类型成就；
3. 总分数榜、单关卡榜、联机对战榜、历史最佳榜；
4. 可选数据可视化 UI（关卡完成情况、胜率曲线）。

## 5.7 特殊目标逻辑

1. 炸弹地鼠 — 击中扣分、短时操作封禁、预警提示；
2. 黄金地鼠 — 高分加成、连击增益、专属声光特效；
3. 五级闯关 — 每关独立配置刷新频率、出现时长、数量与基础速度，联机与单机共用同一套 LEVEL 表。

# 六、采购方案

核心开发板套件（Vela + 屏幕）由实验室提供。扩展物料预算约 ¥62.22：

| 物料       | 型号                 | 数量 | 单价  | 小计    | 渠道    |
|----------|--------------------|----|-----|-------|-------|
| 超声波模块    | HC-SR04            | 1  | ¥5  | ¥5.04 | 淘宝/立创 |
| RGB 灯条   | WS2812B 8灯环        | 1  | ¥12 | ¥10.4 | 淘宝    |
| 工业按键     | 带灯 12mm            | 4  | ¥3  | ¥13.9 | 淘宝    |
| Wi-Fi 模块 | ESP8266<br>ESP-01S | 1  | ¥10 | ¥10.1 | 可选    |

| 物料   | 型号     | 数量 | 单价  | 小计      | 渠道    |
|------|--------|----|-----|---------|-------|
| 外壳材料 | 亚克力/纸板 | 1套 | ¥15 | ¥14.4   | 淘宝    |
| 杜邦线等 | 若干     | 1套 | ¥8  | ¥8.23   | 已有/淘宝 |
| 合计   |        |    |     | ≈¥62.22 |       |

#### 提示

每个硬件模块均有软件降级方案：超声波不可用 → 双按键；灯条不可用 → 板载 LED；Wi-Fi 为可选项。HC-SR04 仅 ¥5，损坏更换成本极低。

## 七、人员分工

### 7.1 团队成员

| 成员       | 学号         | 班级         | 负责模块                       | 备注                 |
|----------|------------|------------|----------------------------|--------------------|
| 张 恒<br>基 | 2024210926 | 2024211301 | 系 统 总 架<br>构 + 全局<br>集成    | 项目负责人              |
| 曹 佳<br>轩 | 2024210994 | 2024211312 | 多 模 态 输<br>入 + 驱动          | 尝试外设实现，辅助前端        |
| 缪钰       | 2024210996 | 2024211303 | 声 光 + GUI<br>重构 + UI       | UI 核心              |
| 郭 志<br>罡 | 2024210937 | 2024211302 | 关 卡 体 系 +<br>AI + 特殊<br>地鼠 | 项目核心框架搭建，代码主要书写    |
| 张 耀<br>辉 | 2024211045 | 2024211302 | Wi-Fi 联机<br>+ versus       | versus 核心实现        |
| 朱 辰<br>骏 | 2024211039 | 2024211301 | 存 储 + 排<br>行 榜 + 成<br>就    | 联机联调 + storage API |

接口规范：开题首日由张恒基编写 `interface.h`，定义打击事件结构体、游戏状态枚举与各模块函数签名。Git 采用 feature 分支模式，每人一个 feature 分支，每日 push，每周一 merge 到 main，冲突由张恒基解决。

## 7.2 张恒基 — 系统总架构 + 全局集成 + 联机联调 + 项目统筹

---

1. 搭建驱动层、中间件层、应用层、AI 层、网络层五层整体系统架构，制定统一编码规范与模块调用规则；
2. 设计全局调度逻辑，实现单机闯关与 Wi-Fi 联机两种主模式切换，解决多模块并发资源冲突问题；
3. 负责 Git 版本管理，制定分支迭代策略、完成代码审核合并、版本打包发布与镜像维护；
4. 编写并维护 `interface.h`，统一输入、声光、关卡、联机、存储对接协议，保障六人并行开发模块兼容；
5. 升级开机菜单与模式框架：闯关选关、联机大厅、排行榜入口、设置页等；
6. 设计系统全局容错机制，处理外设异常、联机失败、UDP 丢包、程序卡顿等问题，优化整机性能；
7. 负责第三周全员集成联调、答辩演示流程设计、双板对战演示脚本与现场问题调试修复；

## 7.3 曹佳轩 — 多模态输入系统 + 事件队列 + 硬件驱动

---

1. 完成 HC-SR04 超声波测距驱动移植与深度优化，实现精度校准、抗干扰过滤、多传感器稳定并发采集；
2. 配置物理按键 K1/K2 中断，采用硬件 + 软件双重防抖，实现短按、长按、连击多类型按键事件识别；
3. 重构统一打击事件队列 (`input_queue.c`)，新增超时保护、事件防丢失与优先级策略；
4. 实现左/右半区按键映射与触摸屏、超声波三路输入融合，输出标准 `hit_event_t`；
5. 适配单机闯关与 Wi-Fi 联机两种模式的输入逻辑，确保联机时各板独立采集、互不干扰；
6. 开发输入精度校准功能，适配 HC-SR04 安装偏差，支持参数持久化；
7. 新增输入外设故障检测机制，传感器断连、按键异常自动检测并上报，支持降级为纯触摸；
8. 可选：实现反转模式 (`MODE_REVERSE`) 玩法逻辑，与主线闯关/联机独立。

## 7.4 缪钰 — 声光反馈 + GUI 全面重构 + 全模块调试

---

1. 开发优化 WS2812B 灯条驱动 (SPI 模拟时序), 实现全彩控色、呼吸/闪烁/流光/彩虹等多模式灯效;
2. 搭建多音效并发播放系统, 整理 hit/combo/bomb/结算/BGM 等资源, 实现混音与音量调节;
3. 设计全场景声光联动, 涵盖连击、关卡切换、联机结算、黄金/炸弹地鼠、游戏结束等特效, 并补充 GPIO LED 击中闪烁降级方案, 保证声光效果可稳定演示;
4. LVGL GUI 全面重构: 统一蓝色主题与 320×480 布局, 重做主菜单、选关、联机大厅、设置与结算页;
5. 排行榜单页面: 总分榜 / 单关榜 / 联机榜 Tab 切换, 调用朱辰骏 `storage.h` 只读展示;
6. 设计联机对战 UI (比分刷新、对局状态、胜负结算页), 与 `versus` 事件回调挂钩;
7. 补齐页面过渡动画、图标、字体层级与触控反馈, 提升答辩演示观感;
8. 负责全模块接入适配, 参与全模块联调, 排查驱动兼容、模块联动、声画不同步等设备调试问题;
9. 实现声光参数持久化 (亮度/音量), 配合外壳集成完成产品化视觉呈现。

## 7.5 郭志罡 — 关卡难度体系 + 特殊地鼠 + 优化触屏逻辑

---

1. 维护 LEVEL 1 - 5 五关参数表, 定义每关刷新间隔、停留时长、同屏数量、速度与特殊地鼠概率;
2. 完善炸弹 (-2 分)、黄金 (+5 分) 地鼠与 COMBO 连击逻辑, 与闯关/联机计分深度挂钩;
3. 设计各关难度曲线与通关解锁条件, 完成闯关流程与联机 LEVEL 一致性验证;
4. 维护击打结果环形缓冲区, 每 5 秒采样命中率、反应时间、连击数, 计算能力分 (0—100);
5. 可选: 在关卡表基础上实现 AI 微调算法, 查表调节地鼠间隔、停留时长与炸弹概率;
6. 统计各关通关数据、玩家表现与特殊目标触发率, 对接朱辰骏存储模块;
7. 参与 `versus` 联机阶段 LEVEL/COMBO/特殊地鼠兼容性验证 (与张耀辉协作)。

## 7.6 张耀辉 — Wi-Fi 联机 + `versus` 模块

---

1. 基于 R528 开发板搭建 UDP Wi-Fi 通信框架, 在 OpenVela/NuttX 上实现 Vela 打地鼠双人联机对战模块 (`versus/` 传输层 + `MyWhackMole.c` 状态机集成);



2. 实现并维护 `MyWhackMole.c` 中 `versus` 联机核心 (`START / SCORE / FINISH / MODE`), 禁止重构状态机; 设计 `START / SCORE / FINISH` 应用层协议, 实现开局、实时分数与对局结束的跨板同步;
3. 实现实时分数同步机制: `VERSUS_MSG_SCORE` 驱动对手分数刷新, 保障双端对战数据一致更新;
4. 引入 `finish_from_peer` 标志位, 解决 `FINISH` 报文循环触发问题, 保证对局状态可靠收敛;
5. 完成 A/B 双板固定 IP 与互补端口配置 (`VERSUS_DEVICE_A/B`), 为实机联机对战提供可运行配置;
6. 双板各自全屏运行, 同步分数、对局启停与 `MODE` 切换, 不做 `LVGL` 分屏;
7. 保障联机与五级 `LEVEL`、黄金/炸弹地鼠、`COMBO` 兼容, 完成多轮双板对战验证;
8. 联机 IP 宏外置, 换板测试仅改配置层参数, 不改 `versus` 核心逻辑。

## 7.7 朱辰骏 — 数据存储 + WiFi 连接可视化 + 排行榜 + 关卡数据

1. 基于 `littlefs` 封装 `storage.c`, 实现多类型数据稳定读写、CRC 校验、掉电保护与并发优化;
2. 实现关卡通关记录、每关最高分、联机对战胜负、游戏时长等持久化, `FINISH` 时自动写入;
3. 提供排行榜数据 API (`storage_get_rank` 等), 供缪钰排行榜单页面调用;
4. 参与双板联调与实机测试, 提供一块开发板, 排查 IP 配置、UDP 通信、状态不同步等问题, 配合张耀辉完成多轮对战验证;
5. 设计排行榜数据结构: 总榜、单关榜、联机榜, 支持 `Top-N` 查询与历史最佳;
6. 实现成就系统: 连击、通关、联机胜率、特殊目标触发等多类型成就判定与存储;
7. 可选: 数据可视化 (关卡完成度、胜率曲线) 与 Wi-Fi 云端同步;
8. 配合缪钰完成排行榜 UI 数据层对接, 配合张恒基完成第三周存储层集成联调。

## 八、项目计划

| 时间                     | 任务                               |
|------------------------|----------------------------------|
| 第 1 周<br>(4.21 - 4.27) | 开题答辩 → 硬件采购 → 各成员搭建开发环境 → 验证基础驱动 |

| 时间                     | 任务                                      |
|------------------------|-----------------------------------------|
| 第 2 周<br>(4.28 - 5.04) | 输入驱动 → 声光驱动 → GUI 重构主框架 → 五关参数表与闯关流程    |
| 第 3 周<br>(5.05 - 5.11) | GUI 视觉定稿 → 联机联调 → 排行榜存储 → 全员集成（重点周）     |
| 第 4 周<br>(5.12 - 5.18) | 压力测试 + Bug 修复 → 外壳组装 → 演示视频 → 结题报告 → 答辩 |

关键里程碑：

1. 4 月 27 日 — 基础 WhackMole 稳定运行，各外设独立验证通过；
2. 5 月 11 日 — 五级闯关 + 双板 Wi-Fi 联机可稳定演示；
3. 5 月 18 日 — 产品形态完整，文档齐备，答辩就绪。

## 九、预期难点与应对

### 9.1 多线程资源竞争

难点：LVGL 渲染、音频播放、传感器轮询多线程并发访问共享状态。

应对：对所有游戏状态变量加 `pthread_mutex`；合理分配各线程栈空间，总量控制在 NuttX Kconfig 上限（327680 字节）内。

### 9.2 模块并行集成冲突

难点：六人同时开发，接口不匹配风险高。

应对：第一天统一定义 `interface.h`；Git feature 分支 + 每日 push + 每周 merge；merge 冲突由张恒基负责解决。

### 9.3 WS2812B 时序精度

难点：WS2812B 信号时序要求  $\pm 150\text{ns}$ ，NuttX 软件模拟可能抖动。

应对：优先 SPI 模拟协议驱动（2.4MHz，110/100 波形编码）；若不稳定，降级为板载 LED 阵列，两套代码提前写好，宏定义切换。

## 十、反转模式（加分项，可选）

### 10.1 玩法设计

一句话概括：屏幕上的锤子自动落下，玩家的手就是地鼠，需把手从 HC-SR04 传感器下方缩回才能躲开。与主线的闯关/联机独立，答辩保底不依赖此模式。

流程： 游戏开始 → 9 个洞之一发光标记玩家位置 → 锤子从上方落向该洞 → HC-SR04 实时检测手部距离 → 探头计分但面临被打风险 → 缩回躲避获分 → 关卡推进锤子加速增多  
核心规则： 必须“探出头”达到一定时间才能得分；锤子落下窗口（约 200ms）内未缩回则扣血（共 3 血）。

### 10.2 传感器逻辑

| 距离       | 状态               |
|----------|------------------|
| 大于 15cm  | 手已缩回 → 地鼠安全      |
| 5 - 15cm | 手在洞口 → 地鼠探头（计分中） |
| 小于 5cm   | 手贴很近 → 完全探出      |

### 10.3 得分机制

```
// 连续在洞口停留 500ms -> +5 分  
// 被击中 -> -1 血  
// 成功躲避 -> +10 分  
// 连续躲避 3 次 -> 分数 x1.5
```

初始锤子落下时间 1.5 秒，随关卡缩短至 0.6 秒。“缩手”为本能动作，实测反应速度优于点屏。一个 HC-SR04 即可判断“手在不在”，无需定位具体洞位。

### 10.4 联机扩展（可选）

若实现反转模式，可沿用双板联机思路：各板独立全屏，一方扮地鼠、一方选落点，仍不做同屏分屏。

## 十一、开题答辩 Q&A 与技术决策

本章汇总开题答辩问答，作为设计依据存档。

### 11.1 关于系统目标

问 你们的“扩展目标”说实现多模态输入融合，具体用户操作体验是什么样的？

答 玩家有三种方式可以打地鼠。第一种是直接用手指点触摸屏，和基础实验一样。第二种是按开发板两侧的物理大按键，左键打屏幕左半区、右键打右半区。第三种最特别——屏幕上方固定一个 HC-SR04 超声波模块，持续检测手部距离。玩家做出“劈砍”动作时，距离从约 25cm 突变到 8cm 以内，程序识别到这个变化就触发一次打击。三路输入在代码里统一抽象成同一种“打击事件”结构体，上层游戏逻辑完全不需要区分来源。

问 AI 难度自适应和基础实验里“随时间加速”有什么本质区别？

答 基础实验是固定曲线——不管你打得好不好，时间到了速度就上去，和玩家表现无关。我们的方案是每 5 秒采样一次玩家的三个指标：最近 10 只地鼠的命中率、平均反应时间（地鼠出现到被击中的毫秒数）、当前连击数。用这三个数加权算出一个能力分，然后动态调整地鼠出现间隔、停留时长和炸弹出现概率。打得好的玩家全程高压不无聊，打得差的玩家不会被虐到弃游。算法本身是几十行 C 代码的加权评分，不涉及大模型，技术上完全可行。

问 “亮点目标”里的声光联动具体是什么效果，不就是灯亮一下吗？

答 不只是亮一下。我们设计了四种状态对应四种灯效：待机状态灯带做蓝色呼吸效果；普通击中时单灯白色闪烁一次；连续击中 3 只以上，灯带切换为橙色流光；游戏结束时整条爆闪彩虹色 3 秒。音效同步配合：普通击中播 hit.wav，连击播专属音效，倒计时最后 5 秒播语音提醒。灯条用 WS2812B 8 灯环，通过 SPI 模拟时序驱动，灯控逻辑在独立线程中运行，和 LVGL 渲染线程解耦。

### 11.2 关于系统架构

问 双板联机用什么通信协议，为什么不用 MQTT 或 HTTP？

答 物理层走 IEEE 802.11 Wi-Fi，网络层 IPv4，传输层用 UDP 数据报 (NuttX POSIX Socket)，应用层是 versus 模块自定义的 START / SCORE / FINISH / MODE 报文。联机只需同步少量游戏状态，UDP 延迟低、实现轻，不需要 HTTP 的请求-响应开销，也不需要 MQTT Broker。板内多线程通信用 POSIX 消息队列传 hit\_event\_t，与板间 Wi-Fi 协议分开设计。

问 你们画了四个线程，NuttX 上真的能同时跑这么多任务吗，会不会内存不够？

答 NuttX 支持多任务，我们在 Kconfig 里已经把栈空间设到 327680 字节，基础实验里也这么配置过。四个线程的分工是：LVGL 主线程负责渲染，优先级最高；音频线程次之，保证声音不卡顿；传感器轮询线程优先级最低，50ms 轮询一次 HC-SR04 足够；文件存储只在游戏结束时写一次，不占用实时资源。线程间通过消息队列传递打击事件，共享的分数变量用 pthread\_mutex 保护，不会出现竞态条件。

问 统一“打击事件”队列怎么实现，三路输入怎么汇聚？

答 定义一个结构体 hit\_event\_t，包含两个字段：来源类型（触摸屏/按键/超声波）和打击位置（x, y 坐标）。触摸屏直接读坐标；物理按键 K1 映射为左区中心坐标，K2 映射为右区中心坐标；超声波检测到突变后，根据当前随机弹出的地鼠位置填入坐标。三路输入各自在自己的处理函数里构造这个结构体，然后 mq\_send 进同一个 POSIX 消息队列。游戏主逻辑只需要 mq\_receive 取事件处理，完全不关心事件来源。

### 11.3 关于采购方案

问 HC-SR04 测距精度够用吗，会不会误触发？

答 HC-SR04 的测量精度是  $\pm 3\text{mm}$ ，检测范围 2cm 到 400cm，完全够用。误触发的问题通过软件滤波解决：要求距离在 100ms 内从大于 15cm 下降到小于 10cm 才算一次有效打击，单次异常读数不触发。实际测试中人手自然挥击的速度远超这个阈值，而手臂缓慢靠近不会触发，区分度很清楚。

问 WS2812B 对时序要求很严，你们有把握在 NuttX 上驱动起来吗，如果驱动不了怎么办？

答 WS2812B 要求信号时序精度  $\pm 150\text{ns}$ ，我们的方案是用 SPI 总线模拟：将 1-bit 数据映射为 SPI 的 3-bit 波形（110 表示逻辑 1，100 表示逻辑 0），SPI 时钟设为 2.4MHz，每个 bit 周期约 417ns，满足时序要求。这个方案在 Linux 平台已经验证可行，NuttX 的 SPI 驱动接口类似。如果实在调不通，备用方案是用板载的 LED1/LED2/LED3 做简化灯效，功能降级但游戏逻辑不受影响。

问 总预算 62 元，万一某个模块买来不好用需要换怎么办？

答 核心贵的模块就一个 HC-SR04，5 块钱，坏了再买一个影响不大。WS2812B 灯环 12 块，也便宜。我们的设计原则是每个硬件模块都有软件降级方案：超声波不可用  $\rightarrow$  退化为双按键操作；灯条不可用  $\rightarrow$  退化为板载 LED；Wi-Fi 本来就是可选项。所以硬件风险不会导致整个项目崩。

## 11.4 关于人员分工

问 六个人并行开发，代码怎么合并，接口谁来定？

答 第一天开题结束后，张恒基负责写一份 `interface.h` 头文件，把所有模块间的接口全部定死——包括打击事件结构体定义、游戏状态枚举、各模块对外暴露的函数签名。所有人基于这份头文件开发自己的模块，只要接口不变，内部实现互不干扰。Git 使用 `feature` 分支模式，每人一个分支，每天 `push`，每周一统一 `merge` 到 `main` 分支，`merge` 冲突由张恒基解决。

问 成员 4 负责 AI 难度引擎，这个算法具体怎么写，有没有过度设计的风险？

答 算法刻意保持简单，就三步：第一步，维护一个长度为 10 的环形缓冲区记录最近 10 次击打结果（命中 1，未命中 0）；第二步，每 5 秒计算命中率和平均反应时间，代入公式算能力分（0 到 100）；第三步，用能力分查一张预设的参数表，直接取出对应的地鼠间隔和停留时长。整个算法估计 80 行 C 代码以内，没有浮点运算密集计算，不会成为性能瓶颈，也不会开发超期。

## 11.5 关于预期难点

问 难点 3 说 WS2812B 时序问题，备用方案是“降级为板载 LED”，这样功能差太多了吧？

答 功能上确实简化了，但游戏体验损失可控。板载有 LED1、LED2、LED3 三个灯，我们可以用它们表示：击中闪烁 LED1、连击时 LED2 常亮、游戏结束三灯同时闪。声音反馈完全不受影响，视觉反馈的核心信息还在，只是没有彩色流光效果。答辩时如果 WS2812B 调通了就用灯条展示，没调通就用板载 LED，两套代码提前都写好，切换只需要改一个宏定义。

问 项目周期只有四周，万一第三周联调出大问题，来不及怎么办？

答 我们按“必做”和“加分”两档规划功能。必做项是：基础游戏 + 物理按键 + 音效 + 本地排行榜，这四个第二周末就能全部完成，不依赖任何新硬件。加分项是：超声波感应 + WS2812B 灯效 + 双人对战，这三个放在第三周冲刺。即使加分项一个都没做完，必做项已经超过基础实验要求，答辩不会翻车。加分项能做几个算几个，做完一个演示一个。

## 11.6 关于反转模式

问 反转模式用一个超声波传感器够吗，只能检测一个位置？



答 够用。HC-SR04 负责检测“手有没有缩回”这一个状态，不需要定位具体在哪个洞——玩家在屏幕上看到自己被分配到哪个洞，然后物理上把手放在传感器下方。锤子落向哪个洞是屏幕上显示的，传感器只判断那个时刻手在不在，逻辑非常清晰，一个传感器完全够。

问 这个模式是不是太难了，玩家反应不过来？

答 难度完全可调。初始锤子落下时间设为 1.5 秒，给够反应时间；随关卡推进缩短到 0.6 秒。而且“缩手”的动作本能反应很快，比点屏幕还快，实测玩家普遍觉得比传统模式更爽。

### 11.7 硬件降级策略汇总

| 故障场景        | 降级方案                   |
|-------------|------------------------|
| 超声波不可用      | 退化为触摸屏 + 双按键           |
| WS2812B 不可用 | 板载 LED1/LED2/LED3 简化灯效 |
| Wi-Fi 不可用   | 仅单机闯关，联机功能不可用          |

## 十二、总结

SmartMole Pro 开题方案以 WhackMole 为基础，在 OpenVela/NuttX 上规划以五级闯关（每关难度不同）与 Wi-Fi 双板联机为核心的嵌入式娱乐系统，并保留多模态输入、声光反馈与数据持久化等扩展能力。不做同屏分屏，双板各自全屏对战。

本报告完成了系统目标、架构设计、模块划分、采购预算、六人职责、四周计划及答辩 Q&A 等开题论证。项目实施进度、分工执行偏差与结题视角的实现分析请分别参阅《任务进度报告》《分工报告》及《结题报告》(conclusion.typ)；开题报告第十四至十七章补充了音效、LED、联机与未成功能的实际实现说明。

#### 说明

开发过程中参照《openvela 快速入门与工程实践（基于 T113S3）》手册进行环境搭建与驱动验证。

## 十三、参考文献

- [1] 百问网. openvela 快速入门与工程实践（基于 T113S3）[M]. Rev. 1.0, 2025.
- [2] Apache Software Foundation. Apache NuttX Real-Time Operating System[EB/OL]. <https://nuttx.apache.org/>, 2024.
- [3] Xiaomi. openvela: Open Source AIoT Operating System[EB/OL]. <https://github.com/open-vela>, 2024.
- [4] LVGL. Light and Versatile Graphics Library Documentation[EB/OL]. <https://docs.lvgl.io/>, 2024.
- [5] WorldSemi. WS2812B Intelligent Control LED Datasheet[Z]. 2020.
- [6] Allwinner Technology. T113-S3 产品 Brief[D]. 全志科技, 2023.

## 十四、音效反馈系统实现

### 14.1 音频资源清单

根目录 `res.zip` 已打包 8 个 wav 文件（解压后部署至板端 `/data/res/`）：

| 音效文件          | 设计触发场景   | 当前代码状态                              |
|---------------|----------|-------------------------------------|
| hit.wav       | 普通击中地鼠   | ✓ 已实现， <code>sound_task</code> L343 |
| gold.wav      | 击中黄金地鼠   | 资源就位，未挂钩                            |
| bomb.wav      | 击中炸弹地鼠   | 资源就位，未挂钩                            |
| combo.wav     | 连击奖励     | 资源就位，未挂钩                            |
| countdown.wav | 倒计时 ≤5 秒 | 资源就位，未挂钩                            |
| gameover.wav  | 游戏结束结算   | 资源就位，未挂钩                            |
| start.wav     | 对局开始     | 资源就位，未挂钩                            |
| bgm.wav       | 背景音乐循环   | 资源就位，未挂钩                            |

### 14.2 技术实现

音效采用独立 `sound_task` 线程（`MyWhackMole.c` L334 - 350），与 LVGL 主线程解耦：

- 请求模型： 游戏逻辑置位 `play_sound_request`（L85、L1071），音频线程 50ms 轮询消费；



2. 播放方式：调用 `system("aplay -D hw:audiocodec /data/res/hit.wav")`，利用 NuttX 板载 audiocodec 设备；
3. 线程创建：`app_create()` 中 `pthread_create(&sound_thread, ..., sound_task, NULL)` (L1138 - 1141)。

```
static void* sound_task(void* arg)
{
    while (1) {
        if (play_sound_request) {
            play_sound_request = 0;
            system("aplay -D hw:audiocodec /data/res/hit.wav");
        }
        usleep(50 * 1000);
    }
    return NULL;
}
```

#### 说明

开题方案中的多音效混音 (BGM + 特效叠加) 尚未实现：当前为单通道顺序播放，扩展方向是在 `sound_task` 中增加音效类型枚举与路径 switch，或改用 `aplay_main()` 非阻塞调用避免阻塞轮询。

## 十五、LED 反馈实现与 WS2812B 降级决策

### 15.1 实际实现

代码使用板载 GPIO LED (`/dev/gpio0`)，非 WS2812B SPI 灯环：

1. 线程： `led_task` (L354 - 381)，20ms 轮询 `led_flash_request`；
2. 灯效： 击中时亮灯 120ms 后熄灭 (`ioctl GPIOC_WRITE 0→1`)；
3. 触发： 与音效同步，在 `mole_click_event` 中置位 `led_flash_request = 1` (L1072)。

### 15.2 降级原因

开题规划的 WS2812B 需 SPI 模拟  $\pm 150\text{ns}$  位时序，在 NuttX 上与 LVGL 渲染、Wi-Fi 联机并行调试周期不足；板载 GPIO LED 驱动路径已在基础实验中验证，故优先保证稳定可演示的声光反馈，将灯环作为可选加分项保留设计方案（见开题 § 5.5）。

## 十六、Wi-Fi 双板联机实现细节

### 16.1 协议栈与模块文件

| 层次 | 实现                                                           |
|----|--------------------------------------------------------------|
| 传输 | UDP/IPv4, <code>versus/versus_wifi_transport.c</code>        |
| 应用 | 自定义报文, <code>versus/versus_protocol.c</code> (CRC + 序列号去重)   |
| 集成 | <code>MyWhackMole.c</code> 中 <code>versus</code> 初始化与 UI 定时器 |

实际联机使用消息类型: `VERSUS_MSG_START`、`VERSUS_MSG_SCORE`、`VERSUS_MSG_FINISH` (见 `versus_protocol.h` L44 - 52)。

### 16.2 线程与 UI 同步

1. 接收线程: `versus_rx_task` (L899 - 943), 独立 `pthread`, 10ms 轮询 `versus_wifi_poll_packet`;
2. UI 定时器: `versus_ui_timer_cb` (L945 - 963), 50ms LVGL 定时器, 处理远端 `START/`  
`FINISH` 与对手分数刷新;
3. 计分显示: P1 为本机 `score_a`, P2 为对手 `score_b` (SCORE 报文经 `update_peer_score` 更新, L965 - 972)。

### 16.3 Wi-Fi 图形连接 (`wifi_ui.c`)

游戏界面 WIFI 按钮 (L659 - 670) 打开 LVGL 弹窗, 底层调用 NuttX `wapi`:

1. `ifup wlan0` → `wapi mode wlan0 2` (STA 模式);
2. `wapi psk wlan0 "<pwd>" 3` → `wapi essid wlan0 "<ssid>" 1`;
3. `renew wlan0` 获取 IP; 扫描结果写入 `/data/wifi_scan.txt`。

### 16.4 IP 锁定问题分析

```
// MyWhackMole.c L1103-1105
config.peer_ip = "192.168.137.91";
config.local_port = 43046;
config.peer_port = 43045;
```

1. 当前限制: 对端 IP 硬编码在 `versus_init_if_enabled()`; 本镜像编译为 `VERSUS_DEVICE_B`, 对端为 A 板;

2. 换板流程： 修改 `peer_ip` / 端口宏 → 重新编译 → 烧录；A/B 板需分别配置互补端口；
3. 未外置化原因： 四周开发周期中联调优先保证双板对战稳定，配置文件 / LVGL 设置页方案已设计（分工报告 P0）但未合入；
4. 解决思路： 启动时从 `/data/versus.conf` 读取 IP/端口；或在 WiFi 弹窗增加「对端 IP」输入框，写入后重启 `versus` 传输层。

## 十七、未实现功能分析

每项按「设计目标 → 未实现原因 → 后续可行性」说明：

### 17.1 HC-SR04 超声波挥击

设计目标： 100ms 距离突变检测，映射为打击坐标。原因： 驱动需 GPIO 时序与滤波调试，人力被联机与 UI 集成占用；源码无 HC-SR04 引用。可行性： 单传感器 + 统一事件队列仍可行，需曹佳轩模块接入。

### 17.2 WS2812B 全彩灯环

设计目标： SPI 2.4MHz 模拟时序，多场景灯效。原因：  $\pm 150\text{ns}$  精度在 NuttX 多线程环境下未调通。可行性： 当前 `led_task` 接口可替换为 WS2812B 驱动，GPIO 方案已可答辩演示。

### 17.3 littlefs 排行榜与成就

设计目标： Top-N 总榜/单关榜/联机榜 + 成就系统。原因： 已实现 `storage.c` 文件持久化（`/data/whackmole_stats.dat`, L6）与 STATS 弹窗（L286 - 322），但未使用 `littlefs` API，亦无排行榜 UI Tab。可行性： `storage_update_result` 已在结算时调用（L850 - 854），扩展字段与读榜 API 即可对接缪钰 UI。

### 17.4 AI 动态难度

设计目标： 每 5 秒采样命中率调节参数。原因： 五级固定参数表（L52 - 58）已满足答辩核心玩法，AI 为开题加分项。可行性： 环形缓冲区与能力分公式已在开题设计，可在 `pop_random_mole` 周期内增量接入。

## 17.5 音效全场景挂钩

---

设计目标： 8 个 wav 按场景触发 + BGM 混音。原因： 音频线程框架已通，仅完成 hit 路径；其余 7 个文件已在镜像资源中。可行性： 在 `mole_click_event` / `update_game_timer` / `start_game` 处分发音效类型即可，工作量约 1-2 人日。

## 17.6 反转模式

---

设计目标： 玩家扮演地鼠躲锤子。原因： 独立玩法模块，依赖多模态输入（挥击检测）就绪；与主线闯关/联机解耦。可行性： 新增 MODE 枚举与独立状态机，不影响 versus 冻结区。